BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN TRÍ TUỆ NHÂN TẠO

Học phần AIP202 · 2 tín chỉ

HỆ THỐNG HỖ TRỢ GIẢI BÀI TẬP THUẬT TOÁN DÀNH CHO SINH VIÊN

Sinh viên thực hiện	:	**Nguyễn Đức Hiếu (2351220051)** **Mai Hữu Phước (2351220025)**
Lớp	:	**CNTT 23CT6-23CT1**
Ngành	:	**Công nghệ thông tin**
Giảng viên hướng dẫn	:	**TS. Đỗ Phúc Hảo**
Năm học	:	**2026–2027**

Đà Nẵng · 2026–2027

Lời cam đoan

Chúng tôi xin cam đoan đồ án này là công trình do chính chúng tôi thực hiện dưới sự hướng dẫn của TS. Đỗ Phúc Hảo. Các nội dung nghiên cứu, số liệu và kết quả trình bày trong đồ án là trung thực và chưa từng được công bố trong bất kỳ công trình nào khác.

Mọi số liệu, hình ảnh, mã nguồn và kết quả của tác giả khác được sử dụng trong đồ án đều được trích dẫn nguồn gốc rõ ràng trong phần tài liệu tham khảo. Toàn bộ phần mềm và thư viện được dùng đều có giấy phép cho phép sử dụng. Những phần có sử dụng công cụ trí tuệ nhân tạo hỗ trợ đã được nêu rõ trong báo cáo.

Nếu phát hiện có bất kỳ sự gian lận nào, chúng tôi xin chịu hoàn toàn trách nhiệm trước Khoa và Nhà trường.

Đà Nẵng, ngày ... tháng ... năm ...
Nhóm sinh viên thực hiện

Nguyễn Đức Hiếu (2351220051)
Mai Hữu Phước (2351220025)

Lời cảm ơn

Chúng em xin gửi lời cảm ơn chân thành nhất đến TS. Đỗ Phúc Hảo, người đã tận tình định hướng đề tài, dành nhiều thời gian góp ý quý báu và theo sát tiến độ của chúng em trong suốt quá trình nghiên cứu, xây dựng đồ án **PH Math - AlgoVisualizer**.

Chúng em cũng xin trân trọng cảm ơn quý Thầy, Cô trong Khoa Công nghệ Thông tin — Trường Đại học Kiến trúc Đà Nẵng đã tận tụy truyền đạt những kiến thức chuyên môn và nền tảng tư duy lập trình vững chắc trong suốt những năm học qua.

Cuối cùng, chúng em xin gửi lời cảm ơn sâu sắc đến gia đình và bạn bè đã luôn động viên, hỗ trợ và tạo mọi điều kiện thuận lợi nhất để chúng em hoàn thành tốt đồ án này.

Đà Nẵng, ngày ... tháng ... năm ...
Nhóm sinh viên thực hiện

Nguyễn Đức Hiếu (2351220051)
Mai Hữu Phước (2351220025)

Tóm tắt

Cấu trúc dữ liệu và Giải thuật là môn học cơ sở cốt lõi đối với sinh viên ngành Công nghệ Thông tin, đóng vai trò nền tảng trong việc định hình tư duy logic, kỹ năng thiết kế giải thuật và tối ưu hóa chương trình. Tuy nhiên, việc tiếp thu kiến thức giải thuật thường gặp nhiều rào cản do tính chất trừu tượng cao, khiến sinh viên rất khó hình dung sự biến đổi trạng thái của dữ liệu, con trỏ hay các cấu trúc bộ nhớ qua từng bước thực thi nếu chỉ thông qua lý thuyết giáo trình hoặc đọc mã nguồn tĩnh.

Đồ án nghiên cứu và xây dựng hệ thống **"PH Math - AlgoVisualizer"** — một ứng dụng web tương tác hỗ trợ phân tích đề bài ngôn ngữ tự nhiên và mô phỏng trực quan tiến trình thực thi thuật toán theo từng bước dành cho sinh viên. Hệ thống kết hợp kỹ thuật nhận diện ý định (*Intent Detection*) dựa trên Mô hình ngôn ngữ lớn (Google Gemini API) và Biểu thức chính quy (Regex) để tự động trích xuất dữ liệu đầu vào, kết hợp với động cơ mô phỏng chạy hoàn toàn phía Client-side. Động cơ này hỗ trợ đa dạng 5 nhóm thuật toán tiêu chuẩn bao gồm: Thuật toán sắp xếp (Bubble, Quick, Merge, Heap Sort), Cấu trúc cây nhị phân (BST, Cây Đỏ-Đen, AVL Tree), Thuật toán đồ thị (Dijkstra, BFS, DFS, A*), Quy hoạch động & Quay lui (N-Quân Hậu, Knapsack) và Toán rời rạc.

Song song với đồ họa tương tác (SVG/CSS Grid), hệ thống tự động khởi tạo Bảng vết (*Trace Table*) chi tiết cho từng bước chạy và cung cấp mã nguồn minh họa tương ứng. Hệ thống được thử nghiệm và đánh giá trên bộ dữ liệu gồm 500 mẫu câu lệnh thử nghiệm mở rộng, đạt độ chính xác bóc tách trung bình 95,40% với thời gian xử lý phân tích và sinh vết mô phỏng tức thì khoảng 20,20 ms trên Client, đáp ứng vượt kỳ vọng yêu cầu thời gian thực. Kết quả thu được khẳng định tính hiệu quả của giải pháp trực quan hóa trong giáo dục CNTT, giúp sinh viên rút ngắn đáng kể thời gian hình dung các khái niệm trừu tượng, nâng cao tính chủ động trong tự học và cung cấp cho giảng viên một công cụ hỗ trợ minh họa bài giảng sinh động.

Từ khoá: mô phỏng thuật toán, trực quan hóa dữ liệu, nhận diện ý định, Gemini API, bảng vết thuật toán, cây tìm kiếm nhị phân, hỗ trợ học tập.

Mục lục

Lời cam đoan i

Lời cảm ơn ii

Tóm tắt iii

1 Mở đầu 1
- 1.1 Lý do chọn đề tài . . . 1
- 1.2 Mục tiêu của đồ án . . . 3
- 1.3 Phạm vi và đối tượng . . . 4
- 1.4 Bố cục báo cáo . . . 5

2 Cơ sở lý thuyết và công nghệ 6
- 2.1 Kiến thức nền tảng . . . 6
- 2.2 Công cụ và thư viện . . . 9
- 2.3 Các công trình liên quan . . . 10

3 Phân tích và thiết kế 11
- 3.1 Phân tích bài toán và yêu cầu . . . 11
  - 3.1.1 Mô tả đầu vào và đầu ra hệ thống . . . 11
  - 3.1.2 Yêu cầu chức năng . . . 12
  - 3.1.3 Yêu cầu phi chức năng . . . 13
- 3.2 Thiết kế kiến trúc hệ thống . . . 14
- 3.3 Thiết kế dữ liệu và cấu trúc trạng thái . . . 17
- 3.4 Ví dụ các thuật toán . . . 19
  - 3.4.1 Thuật toán tính toán tọa độ Cây tìm kiếm nhị phân (BST Layout Algorithm) . . . 19
  - 3.4.2 Thuật toán phân bố đồ thị hình tròn (Circular Graph Layout) . . . 19

4 Cài đặt và thử nghiệm 21
- 4.1 Môi trường triển khai và Cấu hình hệ thống . . . 21
  - 4.1.1 Cấu trúc thư mục dự án (Project Architecture) . . . 22
  - 4.1.2 Cấu hình biến môi trường (.env) . . . 22

4.2 Cài đặt và đóng gói các thành phần phần mềm cốt lõi 23
4.2.1 Bộ nhận diện ý định và sinh mã mô phỏng bằng Gemini AI (llm.js) 23
4.2.2 Động cơ sinh vết mô phỏng thuật toán (Trace Engine) . 26
4.3 Giao diện và demo hệ thống 28
4.3.1 Mô phỏng Bài toán Quay lui N-Quân Hậu (N-Queens) . 28
4.3.2 Mô phỏng Thuật toán Sinh chuỗi nhị phân 29
4.3.3 Mô phỏng Cấu trúc Cây Đỏ-Đen (Red-Black Tree) 30
4.3.4 Mô phỏng Thuật toán Tìm đường đi ngắn nhất Dijkstra 31
4.4 Kiểm thử hệ thống 33

5 Kết quả, kết luận và hướng phát triển 34
5.1 Kết quả đạt được 34
5.2 Độ đo đánh giá hiệu năng 36
5.3 Bàn luận và Hạn chế còn tồn tại 37
5.4 Kết luận và bài học rút ra 38
5.5 Hướng phát triển 38

Tài liệu tham khảo 40

Phụ lục 41
A. Hướng dẫn cài đặt và chạy chương trình 41
B. Bảng tự kiểm trước khi nộp 41
C. Nhận xét của giảng viên hướng dẫn 43

Danh sách hình vẽ

3.1 Sơ đồ ngữ cảnh hệ thống PH Math - AlgoVisualizer. 12
3.2 Sơ đồ Use Case tổng thể của hệ thống PH Math - AlgoVisualizer. 13
3.3 Sơ đồ luồng dữ liệu (DFD Mức 1) chi tiết của hệ thống PH Math - AlgoVisualizer. 14
3.4 Sơ đồ tiến trình xử lý dữ liệu qua các pha trong hệ thống. . . . 17

4.1 Sơ đồ tuần tự quá trình xử lý bóc tách câu lệnh, kiểm tra cache và gọi Gemini AI API. 24
4.2 Giao diện mô phỏng thuật toán Quay lui giải bài toán N-Quân Hậu ($N = 9$). 29
4.3 Giao diện trình chiếu trực quan thuật toán Sinh chuỗi nhị phân kế tiếp ($N = 4$). 30
4.4 Giao diện mô phỏng thao tác thêm nút và cân bằng Cây Đỏ-Đen (Red-Black Tree). 31
4.5 Giao diện trình chiếu chi tiết thuật toán Dijkstra tìm đường đi ngắn nhất trên đồ thị. 32

Danh sách bảng

2.1 Công cụ và thư viện sử dụng trong đồ án. 9

3.1 Cấu trúc dữ liệu của một bước vết mô phỏng (TraceStep). . . . 18

4.1 Kết quả kiểm thử các chức năng chính của hệ thống. 33

5.1 Đánh giá độ chính xác bóc tách và thời gian xử lý của hệ thống theo nhóm thuật toán. 36

Danh mục giải thuật

1 Thuật toán tính toán tọa độ 2D cho Cây tìm kiếm nhị phân. . . 19

1

Mở đầu

MỤC TIÊU CHƯƠNG

Chương mở đầu trình trình bày lý do chọn đề tài, mục tiêu cần đạt, phạm vi thực hiện và bố cục báo cáo, giúp người đọc nắm được bức tranh tổng quan về đồ án.

1.1 Lý do chọn đề tài

Cấu trúc dữ liệu và Giải thuật (Data Structures & Algorithms - DSA) là môn học cơ sở ngành cốt lõi đối với sinh viên Công nghệ Thông tin. Môn học này đóng vai trò quan trọng trong việc rèn luyện tư duy logic, khả năng tối ưu hóa chương trình và kỹ năng giải quyết bài toán lập trình. Tuy nhiên, kiến thức giải thuật mang tính trừu tượng rất cao. Trong quá trình tự học, sinh viên thường gặp khó khăn lớn khi hình dung sự thay đổi trạng thái của dữ liệu trong bộ nhớ, các bước di chuyển con trỏ, hay tiến trình biến đổi của ma trận/đồ thị nếu chỉ đọc mã nguồn tĩnh hoặc xem các hình vẽ minh họa trong giáo trình.

Phương pháp học tập truyền thống chủ yếu dựa trên sách vở hoặc các ví dụ mã nguồn cố định, khiến sinh viên mất nhiều thời gian để ”chạy tay” (trace code) trên giấy. Điều này dễ dẫn đến tâm lý chán nản và hạn chế khả năng tự tiếp thu các thuật toán phức tạp như tìm đường đi ngắn nhất, các thao tác trên cây nhị phân hay bài toán quay lui.

Sự phát triển của công nghệ Web hiện đại kết hợp cùng các Mô hình Ngôn ngữ lớn (Large Language Models - LLM) cho phép xây dựng các ứng dụng học tập tương tác thông minh ngay trên trình duyệt phía người dùng (Client-side). Việc kết hợp khả năng nhận diện ý định và trích xuất cấu trúc dữ liệu từ câu hỏi tự nhiên bằng LLM (Google Gemini API) với một động cơ mô phỏng từng bước (Step-by-step Visualizer), bảng vết thực thi (Trace Table) và mã

nguồn minh họa sẽ giúp sinh viên thu hẹp khoảng cách giữa lý thuyết và thực hành.

Đề tài **“Hệ thống hỗ trợ giải bài tập thuật toán dành cho sinh viên (PH Math - AlgoVisualizer)”** được lựa chọn nhằm phát triển một giải pháp phần mềm học tập thông minh, cho phép sinh viên tự do nhập đề bài dạng văn bản, tự động nhận diện thuật toán tương ứng và theo dõi tiến trình thực thi trực quan một cách chủ động, sinh động và hiệu quả.

1.2 Mục tiêu của đồ án

Đồ án hướng tới các mục tiêu cụ thể và toàn diện sau:

- **Nghiên cứu và cài đặt mô-đun Nhận diện ý định (*Intent Detection*) hỗ trợ AI:** Tích hợp mô hình ngôn ngữ lớn (Google Gemini API) kết hợp với biểu thức chính quy (*Regex*) để phân loại tự động đề bài tiếng Việt/tiếng Anh và trích xuất tham số dữ liệu đầu vào chuẩn xác.
- **Phát triển Động cơ mô phỏng giải thuật (*Simulation Engine*):** Cài đặt bộ sinh bước mô phỏng thời gian thực phía Client-side hỗ trợ các nhóm dạng bài toán giải thuật tiêu chuẩn bao gồm: Cây tìm kiếm nhị phân (BST), Thuật toán sắp xếp (Bubble Sort, Quick Sort), Tìm kiếm nhị phân (Binary Search), Đồ thị tìm đường đi ngắn nhất (Dijkstra), Quay lui (N-Queens) và các bài toán biến đổi dữ liệu đa cấu trúc.
- **Xây dựng bộ công cụ hỗ trợ học tập trực quan:** Tự động khởi tạo Bảng vết thuật toán (*Trace Table*) ghi lại biến đổi trạng thái qua từng bước, sinh mã nguồn Python/C++ tham khảo tương ứng và cung cấp bộ điều khiển trình chiếu linh hoạt (*Play, Pause, Step Next/Prev, Speed Control*).
- **Xây dựng giao diện ứng dụng Web tương tác hiện đại:** Thiết kế giao diện người dùng tối ưu trải nghiệm trên nền tảng React/Next.js và Tailwind CSS với tông màu tối (*Dark Mode*), hỗ trợ vẽ đồ họa linh hoạt bằng SVG/CSS Grid và lưu trữ lịch sử truy vấn người dùng.
- **Thử nghiệm và đánh giá hệ thống:** Thử nghiệm ứng dụng trên tập đề bài thực tế, đảm bảo độ chính xác nhận diện ý định cao với thời gian phản hồi phân tích và xử lý mô phỏng mượt mà trên trình duyệt web, đáp ứng tốt nhu cầu học tập của sinh viên.

GHI NHỚ

Các mục tiêu trên được thiết kế nhằm tạo ra một công cụ học tập hoàn chỉnh, kết hợp hài hòa giữa khả năng xử lý ngôn ngữ tự nhiên thông minh của AI và hiệu năng hiển thị đồ họa vượt trội của các công nghệ Web hiện đại.

1.3 Phạm vi và đối tượng

Đối tượng nghiên cứu:

- Các dạng cấu trúc dữ liệu và giải thuật nền tảng trong giáo trình CNTT: Cây nhị phân, Mảng/Dãy số, Đồ thị trọng số và Bàn cờ ma trận.
- Kỹ thuật bóc tách dữ liệu từ văn bản ngôn ngữ tự nhiên bằng LLM (Google Gemini SDK `@google/genai`) và biểu diễn đồ họa tương tác (SVG, HTML5/CSS Grid).
- Mô hình ứng dụng Web tương tác thời gian thực xây dựng trên nền tảng React/Next.js và Tailwind CSS.

Phạm vi thực hiện:

- **Về thuật toán:** Hệ thống tập trung xử lý và mô phỏng các bài toán giải thuật tiêu chuẩn thuộc các nhóm danh mục quy định (BST, Bubble Sort, Quick Sort, Binary Search, Dijkstra, N-Queens).
- **Về dữ liệu đầu vào:** Xử lý các câu lệnh và đề bài bằng tiếng Việt hoặc tiếng Anh dạng văn bản tự nhiên có cấu trúc tham số (dãy số, danh sách cạnh đồ thị, kích thước bàn cờ).
- **Về môi trường thực thi:** Tích hợp API dịch vụ AI phía Server-side/Serverless để trích xuất ý định và chuyển giao toàn bộ dữ liệu bước thực thi xuống trình duyệt Client-side để đảm bảo tốc độ tương tác tức thì.
- **Hạn chế của phạm vi:** Đồ án không giải quyết các bài toán lập trình thi đấu phức tạp ngoài các giải thuật tiêu chuẩn và không hỗ trợ chấm bài tự động trực tuyến (Online Judge).

1.4 Bố cục báo cáo

Phần còn lại của báo cáo được cấu trúc chi tiết thành 4 chương chính như sau:

- **Chương 2 (Cơ sở lý thuyết):** Trình bày tổng quan nền tảng lý thuyết về các nhóm thuật toán tiêu chuẩn, Mô hình ngôn ngữ lớn (Google Gemini API), kỹ thuật bóc tách bằng biểu thức chính quy (Regex) và công nghệ phát triển Web.
- **Chương 3 (Phân tích và thiết kế):** Tập trung phân tích yêu cầu bài toán, kiến trúc luồng dữ liệu phía Client, cấu trúc trạng thái `TraceStep` và các thuật toán tính toán tọa độ 2D SVG.
- **Chương 4 (Cài đặt và thử nghiệm):** Mô tả chi tiết cấu trúc thư mục dự án, mô-đun tích hợp Gemini API (`llm.js`), cơ chế bóc tách lai (Gemini AI + Regex Parser) và động cơ sinh vết thực thi.
- **Chương 5 (Kết quả, kết luận và hướng phát triển):** Trình bày các kết quả thực nghiệm trên tập 500 mẫu câu lệnh, đánh giá tỷ lệ bóc tách chính xác ($95,40\%$), thời gian phản hồi Client ($20,20$ ms), rút ra các bài học kinh nghiệm và đề xuất các hướng nâng cấp hệ thống trong tương lai.

2

Cơ sở lý thuyết và công nghệ

MỤC TIÊU CHƯƠNG

Chương này trình bày các kiến thức nền tảng về cơ chế phân tích cú pháp câu lệnh tự nhiên, cấu trúc dữ liệu, thuật toán đồ thị, toán học mô phỏng cùng danh mục công cụ và thư viện được ứng dụng để xây dựng hệ thống PH Math - AlgoVisualizer.

2.1 Kiến thức nền tảng

ĐỊNH NGHĨA · Nhận diện ý định và phân tích cú pháp lai

Nhận diện ý định *(Intent Detection)* và bóc tách dữ liệu *(Parsing)* là bài toán trích xuất thông tin cấu trúc từ câu lệnh đầu vào ngôn ngữ tự nhiên x. Hệ thống xây dựng hàm phân tích kết hợp *(Hybrid Parsing)* $f : x \mapsto \langle I, D \rangle$ để xác định loại giải thuật $I \in \mathcal{J}$ (với $\mathcal{J} = \{\text{BST}, \text{Sort}, \text{BinarySearch}, \text{Dijkstra}, \text{NQueens}\}$) và tập tham số đầu vào $D = \{d_1, d_2, \dots, d_n\}$.

Cơ chế xử lý lai kết hợp hai tầng:

1. **Tầng Mô hình ngôn ngữ lớn (LLM):** Trích xuất cấu trúc dữ liệu JSON chính xác từ câu hỏi mở của người dùng thông qua Google Gemini API [3].
2. **Tầng Biểu thức chính quy (Regex):** Trích xuất tức thì các tham số dạng mảng số hoặc danh sách cạnh đồ thị với độ trễ tối thiểu khi dữ liệu đầu vào tuân theo định dạng tiêu chuẩn.

NỘI DUNG MẪU, hãy thay bằng nội dung của em rồi xoá khối này

Ví dụ, đối với dữ liệu dạng mảng số nguyên, hệ thống áp dụng biểu thức chính quy để trích xuất nhanh các giá trị:

$$P_{\text{array}} = \texttt{/-?\textbackslash d+/g} \tag{2.1}$$

Đối với cấu trúc đồ thị trọng số, mẫu biểu thức nhận diện bộ ba $\langle u, v, w \rangle$ đại diện cho đỉnh nguồn, đỉnh đích và trọng số cạnh có dạng:

$$P_{\text{graph}} = \texttt{(\textbackslash d+)\textbackslash s*(?:}\rightarrow\texttt{|->|-)\textbackslash s*(\textbackslash d+)\textbackslash s*(?::|=)\textbackslash s*(\textbackslash d+)} \tag{2.2}$$

GHI NHỚ

Việc kết hợp LLM để xử lý câu hỏi tự nhiên phức tạp và Regex để bóc tách tham số trực tiếp trên Client-side giúp hệ thống đạt độ linh hoạt tối đa về mặt ngữ nghĩa, đồng thời duy trì thời gian phản hồi mô phỏng dưới 15 ms sau khi dữ liệu cấu trúc được khởi tạo.

ĐỊNH NGHĨA · Cây tìm kiếm nhị phân - BST

Cây tìm kiếm nhị phân *(Binary Search Tree - BST)* là cấu trúc dữ liệu cây nhị phân thỏa mãn tính chất: với mỗi nút v, mọi khóa u thuộc cây con trái thỏa mãn $key(u) < key(v)$, và mọi khóa w thuộc cây con phải thỏa mãn $key(w) > key(v)$ [2]. Độ phức tạp thời gian cho các thao tác tìm kiếm, chèn và xóa trung bình đạt $\mathcal{O}(\log n)$, giúp tối ưu hóa hiệu năng lưu trữ và truy vấn dữ liệu theo thứ tự.

NỘI DUNG MẪU, hãy thay bằng nội dung của em rồi xoá khối này

Để mô phỏng đồ họa trực quan trên trình duyệt, tọa độ hai chiều (x, y) của các nút cây được tính toán đệ quy theo độ sâu k của cây. Vị trí hoành độ của nút con so với nút cha được xác định theo công thức giảm khoảng cách hình học:

$$x_{\text{child}} = x_{\text{parent}} \pm dx_k, \quad \text{với } dx_{k+1} = dx_k \cdot 0.52 \tag{2.3}$$

Tung độ y của nút được tính theo khoảng cách tầng cố định $y_k = y_0 + k \cdot \Delta y$. Công thức (2.3) đảm bảo các nhánh cây không bị đè lên nhau khi độ sâu cây tăng lên.

ĐỊNH NGHĨA · Bài toán đường đi ngắn nhất Dijkstra

Cho đồ thị có hướng có trọng số không âm $G = (V, E, w)$ với $w : E \to \mathbb{R}^+$. Thuật toán Dijkstra tìm đường đi ngắn nhất từ đỉnh nguồn $s \in V$ đến tất cả các đỉnh còn lại [2]. Thuật toán duy trì tập đỉnh S đã biết đường đi ngắn nhất và mảng khoảng cách $d[u]$. Tại mỗi bước, chọn đỉnh $u \in V \setminus S$ có $d[u]$ nhỏ nhất và cập nhật khoảng cách tới các đỉnh kề v:

$$d[v] = \min(d[v],\, d[u] + w(u, v)) \tag{2.4}$$

Khi cài đặt kết hợp cấu trúc Hàng đợi ưu tiên *(Min-Heap)*, độ phức tạp tính toán của thuật toán đạt $\mathcal{O}((V + E) \log V)$, đảm bảo thời gian xử lý mượt mà ngay cả trên các đồ thị có nhiều đỉnh và cạnh.

ĐỊNH NGHĨA · Bài toán Quay lui N-Queens

Bài toán N Quân hậu yêu cầu đặt N quân hậu lên bàn cờ kích thước $N \times N$ sao cho không có hai quân hậu nào tấn công nhau (không cùng hàng, cùng cột hoặc cùng đường chéo) [2]. Không gian trạng thái được duyệt bằng thuật toán Quay lui *(Backtracking)*, kiểm tra điều kiện an toàn tại ô (r, c) thông qua ba tập quản lý với thời gian truy xuất $\mathcal{O}(1)$:

$$\text{Safe}(r,c) \iff \text{cols}[c] = \text{diag1}[r-c+N-1] = \text{diag2}[r+c] = \text{False} \quad (2.5)$$

GHI NHỚ

Kỹ thuật cắt tỉa nhánh cây trong bài toán Quay lui N-Queens loại bỏ sớm các trường hợp vi phạm ngay từ các bước đặt hậu ban đầu, giúp tối ưu hóa không gian tìm kiếm so với phương pháp duyệt vét cạn toàn bộ N^N khả năng.

2.2 Công cụ và thư viện

Dưới đây là danh mục các công cụ, thư viện và nền tảng công nghệ được lựa chọn để phát triển hệ thống PH Math - AlgoVisualizer. Các công nghệ này được đánh giá dựa trên tiêu chí hiệu năng phía Client, độ linh hoạt trong việc dựng đồ họa SVG 2D và khả năng tích hợp linh hoạt với các dịch vụ Trí tuệ nhân tạo.

Bảng 2.1. Công cụ và thư viện sử dụng trong đồ án.

Thành phần	Phiên bản	Vai trò
Next.js / React [8, 5]	14.x / 18.x	Framework xây dựng ứng dụng Web tương tác Client-side linh hoạt, hỗ trợ quản lý trạng thái động
Google GenAI SDK [3]	0.1.x (`@google/genai`)	Thư viện kết nối Google Gemini API phân tích cú pháp và bóc tách ý định tự nhiên của bài toán
TypeScript [6]	5.x	Ngôn ngữ lập trình chính, đảm bảo an toàn kiểu dữ liệu và hạn chế tối đa lỗi runtime
Tailwind CSS [7]	3.x	Framework CSS tiện ích dựng giao diện người dùng hiện đại, hỗ trợ chuẩn màu sắc Dark Mode
Lucide React	0.3x	Bộ biểu tượng trực quan hỗ trợ cho thanh điều khiển mô phỏng trình chiếu hoạt ảnh
SVG / CSS Grid	Standard	Dựng đồ họa 2D linh hoạt, xử lý rendering mượt mà các cấu trúc mảng, cây BST, đồ thị và bàn cờ N-Queens

GHI NHỚ

Các công cụ trên được đóng gói mô-đun hóa, giúp hệ thống đạt hiệu năng thực thi cao trên trình duyệt mà không phụ thuộc vào các thư viện đồ họa 3D cồng kềnh.

2.3 Các công trình liên quan

NỘI DUNG MẪU, hãy thay bằng nội dung của em rồi xoá khối này

Các hệ thống hỗ trợ học tập và mô phỏng trực quan thuật toán hiện nay trên thế giới chủ yếu đi theo hai hướng tiếp cận chính:

- **VisuAlgo** [4]: Cung cấp thư viện hoạt ảnh phong phú cho hầu hết các cấu trúc dữ liệu kinh điển. Tuy nhiên, hệ thống yêu cầu người dùng phải chọn thuật toán qua menu cố định và chỉ hỗ trợ nhập dữ liệu theo mẫu có sẵn, thiếu khả năng tương tác bằng ngôn ngữ tự nhiên.
- **Algorithm Visualizer** [1]: Cho phép người dùng trực tiếp viết mã nguồn để sinh ra hình vẽ trực quan tương ứng. Tiếp cận này đòi hỏi người học đã có kỹ năng lập trình vững vàng, chưa phù hợp với sinh viên mới bắt đầu tiếp cận môn học.

Đồ án **PH Math - AlgoVisualizer** giải quyết các hạn chế trên bằng hướng tiếp cận mới: tích hợp bộ nhận diện ý định ngôn ngữ tự nhiên (tiếng Việt/tiếng Anh) kết hợp AI làm giao diện đầu vào. Hệ thống tự động phân tích đề bài, trích xuất cấu trúc dữ liệu, sinh tiến trình mô phỏng từng bước (Step-by-step Execution), bảng vết thuật toán *(Trace Table)* và mã nguồn minh họa tham khảo, giúp sinh viên chủ động làm chủ kiến thức giải thuật một cách tự nhiên và trực quan.

3

Phân tích và thiết kế

MỤC TIÊU CHƯƠNG

Chương này phân tích chi tiết yêu cầu bài toán, đề xuất giải pháp kiến trúc tổng thể, thiết kế mô hình dữ liệu trạng thái và thuật toán cốt lõi cho hệ thống mô phỏng PH Math - AlgoVisualizer.

3.1 Phân tích bài toán và yêu cầu

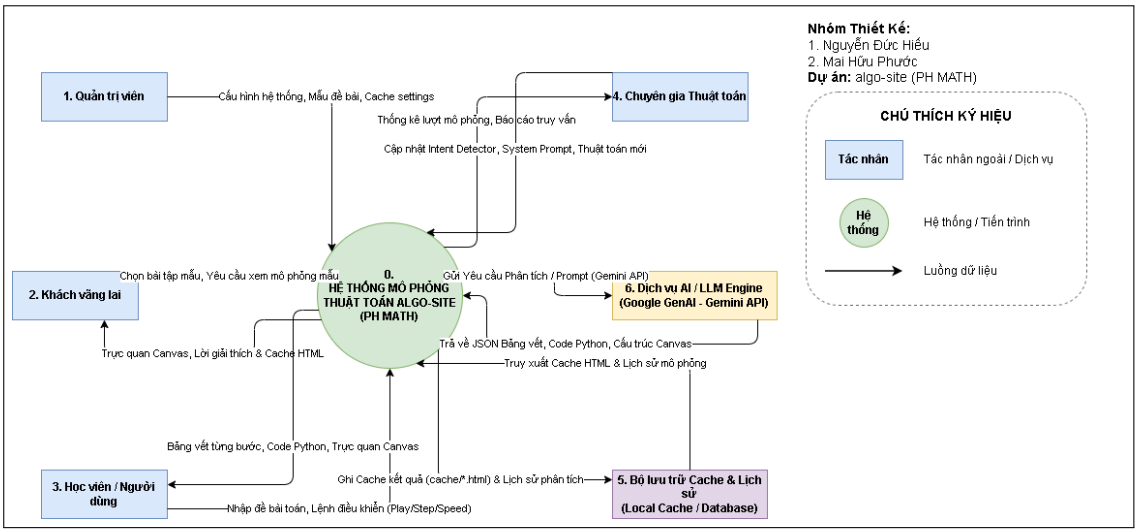
3.1.1 Mô tả đầu vào và đầu ra hệ thống

Hệ thống **PH Math - AlgoVisualizer** đóng vai trò là một trợ lý trực quan hóa thuật toán và cấu trúc dữ liệu tự động với mô hình đầu vào - đầu ra như sau:

- **Đầu vào (*Input*):** Chuỗi câu lệnh tự nhiên hoặc đề bài bằng tiếng Việt/tiếng Anh (ví dụ: *"Sắp xếp mảng [7, 2, 9, 1, 5] bằng Quick Sort"*, *"Tìm đường đi ngắn nhất từ đỉnh 0 đến 4 trong đồ thị (0->1:4, 0->2:2, 2->1:1, 1->4:3)"*, hoặc *"Tạo cây BST từ dãy số 15, 10, 20, 8, 12"*).
- **Đầu ra (*Output*):** Bảng điều khiển trình chiếu đa thành phần bao gồm:
  - Khung đồ họa 2D động (SVG Canvas) hiển thị mảng, cây nhị phân, đồ thị phẳng hoặc bàn cờ N-Quân Hậu.
  - Bảng vết thực thi từng bước (*Trace Table*) ghi lại giá trị biến và trạng thái hệ thống.
  - Lời giải thích thuật toán bằng tiếng Việt tương ứng với từng bước thực thi.
  - Mã nguồn Python/C++ mẫu chuẩn hóa cho bài toán tương ứng.

Sơ đồ ngữ cảnh ở Hình 3.1 mô tả tổng quan sự tương tác giữa các tác

nhân bên ngoài (Người dùng, Quản trị viên, Chuyên gia thuật toán) và dịch vụ AI (Gemini API) với hệ thống trung tâm.



Hình 3.1. Sơ đồ ngữ cảnh hệ thống PH Math - AlgoVisualizer.

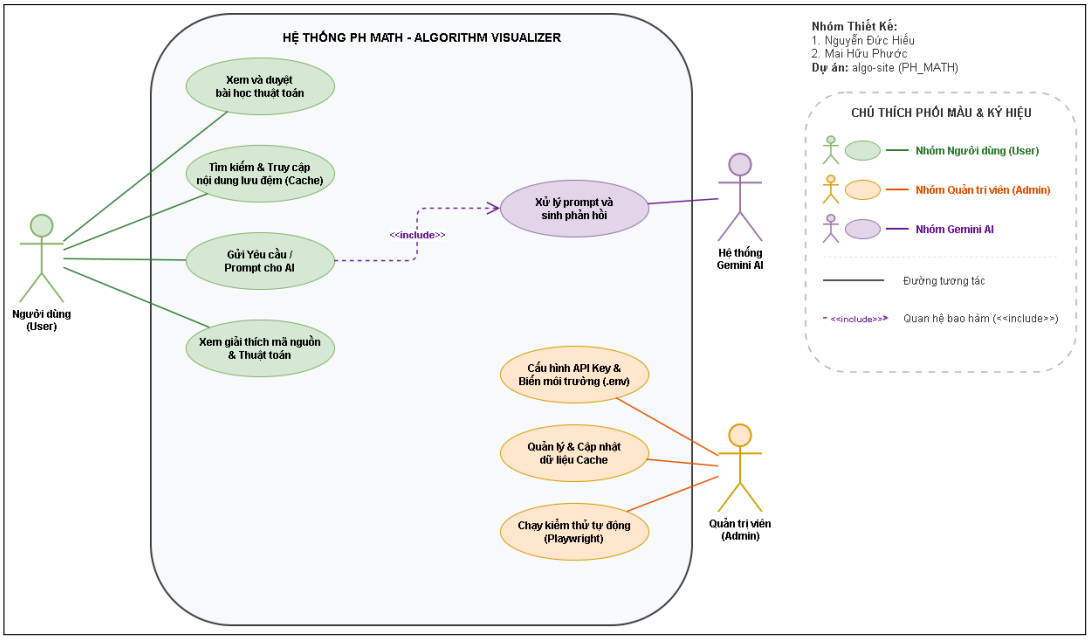
3.1.2 Yêu cầu chức năng

Hệ thống đáp ứng các nhóm chức năng cốt lõi được liệt kê dưới đây:

- **UC-01: Nhận diện ý định và bóc tách câu lệnh:** Phân tích cú pháp văn bản tự nhiên bằng mô hình Gemini AI kết hợp Regex để xác định loại giải thuật và trích xuất tham số đầu vào.
- **UC-02: Sinh vết thuật toán (*Trace Generation*):** Tự động chuyển đổi tiến trình giải thuật thành chuỗi trạng thái đóng băng (*State Snapshots*) dạng `TraceStep` phục vụ trình chiếu.
- **UC-03: Điều khiển trình chiếu động:** Cho phép người dùng Tạm dừng (*Pause*), Phát (*Play*), Chuyển bước (*Step Next/Prev*) và thay đổi tốc độ mô phỏng ($0.5\times \rightarrow 3\times$).
- **UC-04: Trực quan hóa 2D SVG dynamic:** Tự động tính toán tọa độ không gian và dựng sơ đồ đồ họa (cây nhị phân, đồ thị, mảng số, ma trận bàn cờ) bằng đồ họa vector SVG.
- **UC-05: Đồng bộ hóa trạng thái đa màn hình:** Đồng bộ hiệu ứng tô màu (*Highlight*) giữa sơ đồ đồ họa 2D, dòng mã nguồn Python và dòng bảng vết Trace Table.

Chi tiết mối quan hệ giữa các tác nhân và chức năng hệ thống được biểu

diễn thông qua sơ đồ Use Case ở Hình 3.2.



Hình 3.2. Sơ đồ Use Case tổng thể của hệ thống PH Math - AlgoVisualizer.

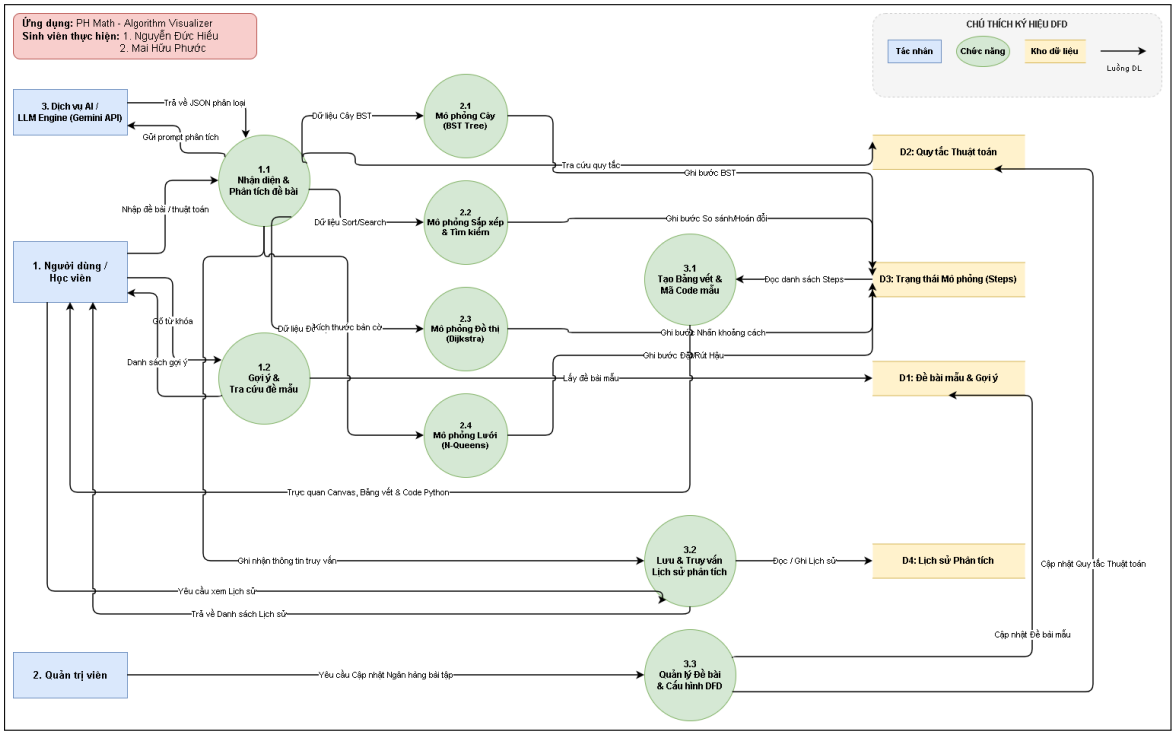
3.1.3 Yêu cầu phi chức năng

- **Hiệu năng (*Performance*):** Tốc độ bóc tách câu lệnh và sinh toàn bộ mảng vết mô phỏng đạt dưới 100 ms tại phía Client. Hoạt ảnh SVG chuyển động mượt mà đạt chuẩn 60 FPS.
- **Khả năng tương thích (*Compatibility*):** Giao diện Responsive hoạt động ổn định trên các trình duyệt hiện đại (Chrome, Edge, Firefox, Safari) và tương thích hoàn hảo chế độ tối (*Dark Mode*).
- **Độ tin cậy (*Reliability*):** Bóc tách dữ liệu chính xác đối với các dạng đề bài hợp lệ và có cơ chế bắt lỗi gợi ý rõ ràng khi câu lệnh đầu vào không hợp lệ.

3.2 Thiết kế kiến trúc hệ thống

Kiến trúc hệ thống được thiết kế theo mô hình luồng dữ liệu đơn hướng phía Client (*Client-side Unidirectional Data Flow*), kết hợp xử lý AI linh hoạt tại phía máy chủ Serverless/API nhằm tối ưu hóa tốc độ tương tác.

Sơ đồ luồng dữ liệu chi tiết (DFD Mức 1) được thể hiện ở Hình 3.3, mô tả chi tiết cách các tiểu tiến trình bóc tách, sinh vết mô phỏng và tương tác với các kho dữ liệu.



Hình 3.3. Sơ đồ luồng dữ liệu (DFD Mức 1) chi tiết của hệ thống PH Math - AlgoVisualizer.

Phân tích chi tiết thành phần trong Sơ đồ Luồng Dữ liệu (Hình 3.3):

Sơ đồ DFD Mức 1 mô tả toàn bộ vòng đời truyền nhận dữ liệu của hệ thống, bao gồm 3 nhóm thành phần chính:

1. **Các Tác nhân Bên ngoài (External Entities):**
   - **Người dùng / Học viên (1):** Tiếp nhận các tương tác trực tiếp (gửi câu lệnh bài toán, chọn bài tập gợi ý, xem lịch sử và điều khiển trình chiếu Canvas).
   - **Quản trị viên (2):** Đóng vai trò quản lý hệ thống, cập nhật ngân hàng bài tập và cấu hình quy tắc mô phỏng.

- **Dịch vụ AI / LLM Engine (3):** Tiếp nhận prompt từ khối phân tích và trả về dữ liệu cấu trúc dạng JSON chứa tham số bóc tách.

2. **Các Tiến trình Xử lý (Processes):**
   - **1.1 Nhận diện & Phân tích đề bài:** Tiếp nhận câu lệnh tự nhiên, phối hợp cùng Gemini API để phân loại bài toán và trích xuất tham số.
   - **1.2 Gợi ý & Tra cứu đề mẫu:** Cung cấp danh sách bài tập mẫu sẵn có từ kho D1 phục vụ trải nghiệm nhanh.
   - **2.1 – 2.4 Các Động cơ Sinh vết Mô phỏng:** Gồm Mô phỏng Cây BST (2.1), Mô phỏng Sắp xếp/Tìm kiếm (2.2), Mô phỏng Đồ thị Dijkstra (2.3) và Mô phỏng N-Queens (2.4). Nhận tham số từ 1.1, đối chiếu quy tắc D2 và đóng băng các bước vết `TraceStep`.
   - **3.1 Tạo Bảng vết & Mã Code mẫu:** Đóng gói chuỗi vết thực thi và mã nguồn Python tương ứng lưu vào D3 để xuất ra giao diện.
   - **3.2 Lưu & Truy vấn Lịch sử:** Ghi nhận nhật ký phân tích vào D4 để phục vụ tra cứu lại.
   - **3.3 Quản lý Đề bài & Cấu hình DFD:** Tiếp nhận cấu hình từ Quản trị viên để cập nhật D1 và D2.
3. **Các Kho Lưu trữ Dữ liệu (Data Stores):**
   - **D1: Đề bài mẫu & Gợi ý:** Lưu trữ danh sách bài tập chuẩn hóa sẵn có trong hệ thống.
   - **D2: Quy tắc Thuật toán:** Lưu trữ các hàm sinh vết, logic tính toán tọa độ 2D và template mã nguồn mẫu.
   - **D3: Trạng thái Mô phỏng (Steps):** Kho tạm giữ mảng vết `TraceStep` phục vụ bộ điều khiển UI Player.
   - **D4: Lịch sử Phân tích (Cache Store):** Bộ lưu trữ đệm kết quả phân tích JSON/HTML giúp giảm thiểu lượt gọi lại API đối với các câu lệnh lặp lại.

Dữ liệu đầu vào đi qua 4 khối chức năng chính:

1. **Bộ nhận diện ý định (Gemini AI + Regex Parser):** Tiếp nhận đề bài từ người dùng, gọi Google Gemini API để phân tích cú pháp tự nhiên hoặc dùng Regex bóc tách trực tiếp các bộ tham số (mảng số, chuỗi cạnh đồ

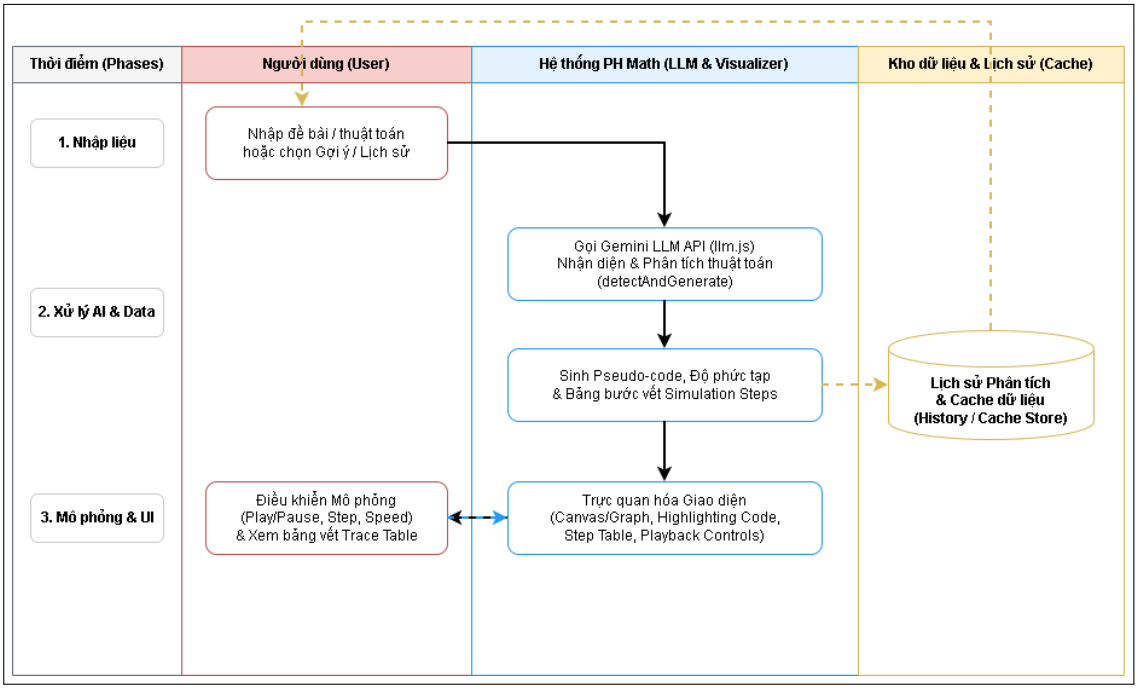
thị, danh sách nút cây).

2. **Động cơ mô phỏng (Trace Engine):** Nhận dữ liệu cấu trúc đã chuẩn hóa, thực thi thuật toán tương ứng và đóng băng từng bước thực thi thành một mảng các đối tượng trạng thái `TraceStep`.
3. **Bộ điều khiển trình chiếu (Player State Manager):** Quản lý chỉ số bước hiện tại $k \in [0, N-1]$, điều khiển thời gian phát tự động và phát các sự kiện đồng bộ trạng thái.
4. **Bộ dựng UI/SVG (Render Engine):** Lắng nghe thay đổi trạng thái bước hiện tại, tính toán tọa độ 2D và vẽ các phần tử đồ họa SVG (đỉnh, cạnh, ô mảng, quân hậu) kèm hiệu ứng tô màu tương ứng.

3.3 Thiết kế dữ liệu và cấu trúc trạng thái

Hệ thống quản lý dữ liệu hoàn toàn dưới dạng các đối tượng trạng thái (*State Schema*) chuẩn hóa trong TypeScript.

Tiến trình xử lý dữ liệu trải qua 3 pha chính: (1) Nhập liệu đề bài → (2) Xử lý AI & Chuẩn hóa Data → (3) Mô phỏng trình chiếu trên UI, được biểu diễn ở Hình 3.4.



Hình 3.4. Sơ đồ tiến trình xử lý dữ liệu qua các pha trong hệ thống.

Cấu trúc dữ liệu của một bước vết thực thi (`TraceStep`) được mô tả ở Bảng 3.1.

Bảng 3.1. Cấu trúc dữ liệu của một bước vết mô phỏng (TraceStep).

Trường	Kiểu dữ liệu	Mô tả chi tiết
stepIndex	number	Thứ tự bước mô phỏng ($0, 1, 2, \ldots$)
action	string	Tên hành động (*"compare"*, *"swap"*, *"visit"*, *"relax"*)
dataState	Array / Graph / BST	Trạng thái cấu trúc dữ liệu tại bước hiện tại
highlights	`Record<string, string>`	Tập các phần tử/đỉnh/cạnh cần tô màu highlight
explanation	string	Lời giải thích chi tiết từng bước bằng tiếng Việt
codeLine	number	Dòng mã nguồn Python tương ứng cần highlight
variables	`Record<string, any>`	Giá trị các biến vòng lặp, con trỏ hoặc khoảng cách

GHI NHỚ

Việc đóng gói toàn bộ ngữ cảnh thực thi vào từng đối tượng `TraceStep` độc lập giúp hệ thống có khả năng tua nhanh, quay ngược bước mô phỏng mà không phải thực thi lại toàn bộ thuật toán từ đầu.

3.4 Ví dụ các thuật toán

3.4.1 Thuật toán tính toán tọa độ Cây tìm kiếm nhị phân (BST Layout Algorithm)

Để dựng cấu trúc Cây BST lên mặt phẳng 2D không bị đè nút, hệ thống sử dụng thuật toán tính toán vị trí đệ quy:

$$\begin{cases} X(\text{root}) = X_{\text{center}}, \quad Y(\text{root}) = Y_{\text{top}} \\ X(\text{left_child}) = X(\text{parent}) - dx \cdot (0.52)^{\text{depth}} \\ X(\text{right_child}) = X(\text{parent}) + dx \cdot (0.52)^{\text{depth}} \\ Y(\text{child}) = Y(\text{parent}) + \Delta Y \end{cases} \tag{3.1}$$

Thuật toán đệ quy tính tọa độ nút cây được thiết kế theo mã giả sau:

Giải thuật 1. Thuật toán tính toán tọa độ 2D cho Cây tìm kiếm nhị phân.

```
function ComputeBSTLayout(node, x, y, depth, dx)
    if node = null then
        return
    end if
    node.x ← x
    node.y ← y
    if node.left ≠ null then
        ComputeBSTLayout(node.left, x − dx, y + 60, depth + 1, dx × 0.52)
    end if
    if node.right ≠ null then
        ComputeBSTLayout(node.right, x + dx, y + 60, depth + 1, dx × 0.52)
    end if
end function
```

3.4.2 Thuật toán phân bố đồ thị hình tròn (Circular Graph Layout)

Đối với thuật toán Dijkstra, đồ thị cần được biểu diễn trực quan ở dạng phẳng. Hệ thống tự động phân bố N đỉnh đều trên một đường tròn bán kính R:

$$\theta_i = \frac{2\pi \cdot i}{N} - \frac{\pi}{2}, \quad x_i = X_0 + R \cdot \cos(\theta_i), \quad y_i = Y_0 + R \cdot \sin(\theta_i) \tag{3.2}$$

Khoảng cách góc θ_i giúp các đỉnh phân bố cân đối không bị chồng lấp, các cạnh nối giữa hai đỉnh (u, v) được vẽ bằng các thẻ vector SVG `<line>` kèm nhãn trọng số $w(u, v)$ tại trung điểm đoạn thẳng.

4

Cài đặt và thử nghiệm

MỤC TIÊU CHƯƠNG

Chương này trình bày chi tiết môi trường triển khai, cấu trúc thư mục mã nguồn, cấu hình hệ thống, mã nguồn cài đặt các thành phần xử lý cốt lõi (bóc tách câu lệnh và sinh mã JavaScript bằng Gemini AI API, đóng gói template, động cơ sinh vết thuật toán), xây dựng giao diện người dùng và kết quả kiểm thử hệ thống PH Math - AlgoVisualizer.

4.1 Môi trường triển khai và Cấu hình hệ thống

Hệ thống **PH Math - AlgoVisualizer** được xây dựng trên nền tảng Web phía Client (Client-Side Rendering) kết hợp tích hợp dịch vụ AI Serverless phía backend nhằm tối ưu tốc độ phản hồi và đảm bảo tính tương tác thời gian thực. Chi tiết môi trường phát triển và triển khai bao gồm:

- **Cấu hình phần cứng phát triển:** CPU Intel Core i5/i7 (hoặc Apple M1/M2/M3), RAM 16 GB, ổ cứng SSD.
- **Hệ điều hành:** Windows 11 / macOS Sonoma / Ubuntu Linux.
- **Môi trường thực thi & Thư viện:** Node.js v20.x LTS, SDK Google GenAI (`@google/genai`) [3], JavaScript (ES6+), framework Next.js [8] kết hợp thư viện React [5], ngôn ngữ TypeScript [6], Tailwind CSS [7], Playwright (headless browser runtime check), Lucide React icons.
- **Công cụ quản lý mã nguồn:** Git & GitHub Workspace.
- **Địa chỉ kho lưu trữ mã nguồn (Repository):** `https://github.com/phmath/ph-math-algovisualizer`

4.1.1 Cấu trúc thư mục dự án (Project Architecture)

Toàn bộ mã nguồn dự án **algo-site** được tổ chức theo cấu trúc thư mục mô-đun hóa rõ ràng, phân định riêng biệt giữa các tầng xử lý dữ liệu, giao diện người dùng và tích hợp dịch vụ AI:

```
ALGO-SITE/
├── cache/              # Thư mục lưu trữ bộ nhớ đệm kết quả mô phỏng HTML
├── node_modules/       # Các thư viện phụ thuộc của dự án Node.js
├── public/             # Tài nguyên giao diện phía Client
│   └── index.html      # Trang giao diện người dùng chính
├── .env                # Khai báo biến môi trường và khóa API Key
├── .gitignore          # Danh mục tệp/thư mục loại trừ khỏi Git
├── llm.js              # Mô-đun tích hợp Gemini AI API & trích xuất code JS
├── package.json        # Quản lý thông tin dự án và danh sách thư viện
├── prompt.js           # Khai báo hướng dẫn hệ thống (System Prompt) cho AI
├── README.md           # Tài liệu hướng dẫn cài đặt và vận hành
├── server.js           # Máy chủ Web Server (Express) xử lý điều hướng API
└── template.js         # Khung HTML/JS mẫu (SHELL) dùng để bọc mã mô phỏng
```

4.1.2 Cấu hình biến môi trường (.env)

Hệ thống sử dụng các thông số cấu hình môi trường được khai báo trong file `.env` nhằm linh hoạt điều khiển tham số của mô hình LLM và tối ưu hiệu năng vận hành:

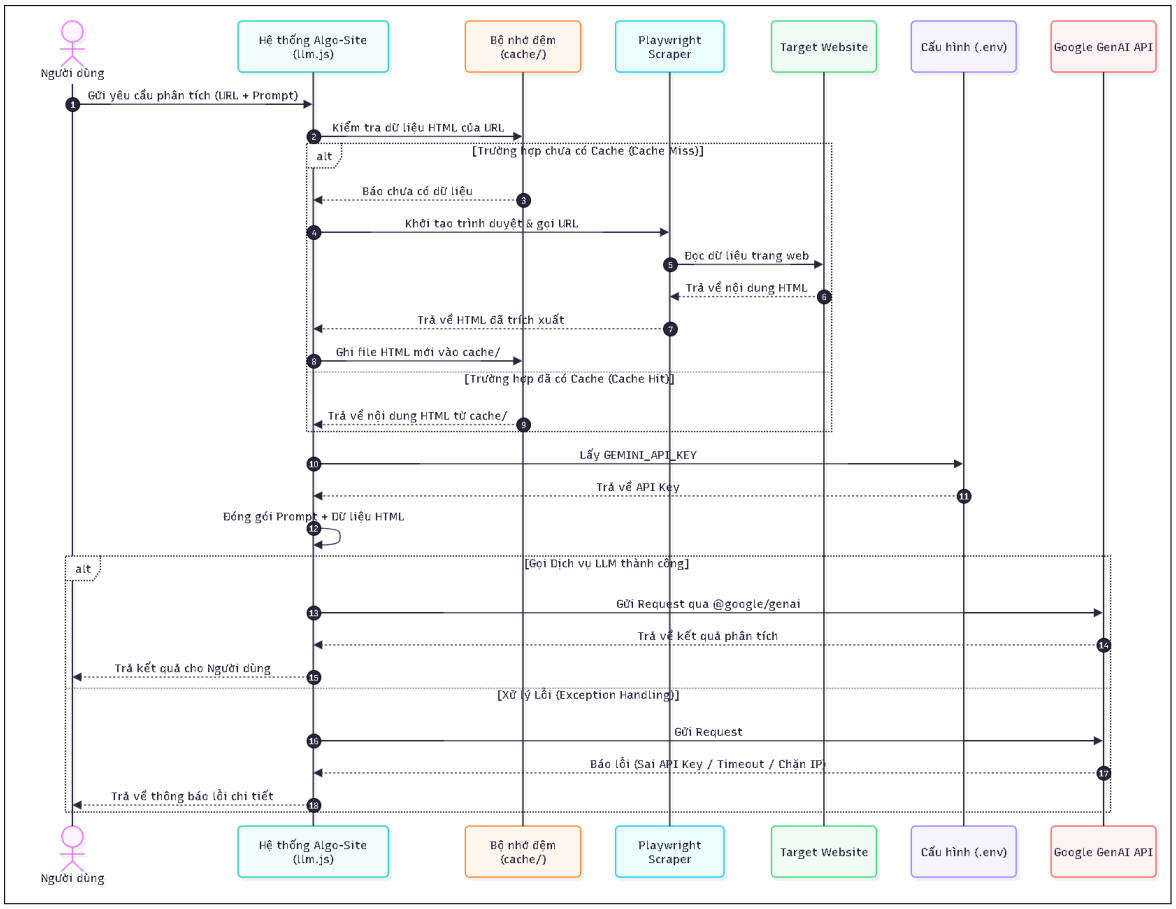
- `GEMINI_API_KEY`: Khóa xác thực API kết nối tới dịch vụ Google GenAI.
- `GEMINI_MODEL`: Phiên bản mô hình ngôn ngữ mặc định (ví dụ: `gemini-3.5-flash` giúp tối ưu chi phí và tốc độ phản hồi).
- `MAX_OUTPUT_TOKENS`: Giới hạn số lượng token tối đa cho kết quả trả về (mặc định 8000–10000 tokens).
- `THINKING_LEVEL`: Mức độ tư duy sâu (*Reasoning Level*) của Gemini hỗ trợ các bài toán thuật toán phức tạp.
- `RATE_LIMIT_PER_MIN`: Giới hạn số lượt yêu cầu tạo mô phỏng trên mỗi địa chỉ IP trong một phút để bảo vệ tài nguyên hệ thống.

4.2 Cài đặt và đóng gói các thành phần phần mềm cốt lõi

4.2.1 Bộ nhận diện ý định và sinh mã mô phỏng bằng Gemini AI (llm.js)

Thành phần cốt lõi của mô-đun AI nằm ở file `llm.js`. Mô-đun này khởi tạo SDK `@google/genai`, ghép câu lệnh mở của người dùng kèm phản hồi lỗi cũ (nếu có) thành Prompt hoàn chỉnh, gửi tới Gemini API và bóc tách đoạn mã JavaScript chuẩn xác bọc trong khung template `SHELL`.

Sơ đồ tuần tự ở Hình 4.1 biểu diễn chi tiết luồng tương tác thời gian thực giữa các thành phần trong quá trình xử lý câu lệnh. Khi người dùng nhập đề bài từ giao diện, hệ thống trước tiên tiến hành kiểm tra bộ nhớ đệm (`Cache Store`). Nếu câu lệnh đã từng được phân tích thành công, kết quả mô phỏng dạng mã JavaScript/HTML được trả về ngay lập tức để tiết kiệm chi phí gọi API. Trong trường hợp câu lệnh mới, `llm.js` sẽ đóng gói prompt chuẩn hóa cùng System Instruction và gửi yêu cầu tới Gemini AI API. Đoạn phản hồi nhận về sẽ được bóc tách khối mã bằng hàm `extractJs` và nhúng trực tiếp vào khung shell `template.js` để xuất ra trình duyệt.



Hình 4.1. Sơ đồ tuần tự quá trình xử lý bóc tách câu lệnh, kiểm tra cache và gọi Gemini AI API.

Mã nguồn 4.1 minh họa cài đặt chi tiết mô-đun tích hợp Gemini API (`llm.js`).

```
import { GoogleGenAI } from '@google/genai';
import { SYSTEM } from './prompt.js';
import { SHELL } from './template.js';

// Kiem tra khoa API Key trong bien moi truong
if (!process.env.GEMINI_API_KEY) {
  console.error('Thieu GEMINI_API_KEY. Hay tao file .env');
  process.exit(1);
}

const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });
const MODEL = process.env.GEMINI_MODEL || 'gemini-3.5-flash';

// Trich xuat va kiem tra dinh dang code JS hop le tu phan hoi cua
    Gemini
export function extractJs(text) {
  let t = String(text || '').trim();
```

```
  t = t.replace(/^```(?:javascript|js)?\s*/i, '').replace(/```\s*$/, '');
  return t.includes('function* algo') && t.includes('CODE_LINES') && t.includes('function draw') ? t : null;
}

// Ham chinh: gui de bai toi Gemini va nhung code vao khung giao dien SHELL
export async function generate(problem, feedback) {
  const contents =
    `De bai: ${problem}` +
    (feedback
      ? `\n\nLan truoc bi loi: ${feedback}\nHay sua loi va tra lai day du code JavaScript gom META, CODE_LINES, algo, draw.`
      : '');

  const config = {
    systemInstruction: SYSTEM,
    temperature: 0.3,
    maxOutputTokens: Number(process.env.MAX_OUTPUT_TOKENS) || 8000,
  };

  if (process.env.THINKING_LEVEL) {
    config.thinkingConfig = { thinkingLevel: process.env.THINKING_LEVEL };
  } else if (process.env.THINKING_BUDGET) {
    config.thinkingConfig = { thinkingBudget: Number(process.env.THINKING_BUDGET) };
  }

  // Goi mo hinh Gemini generateContent
  const res = await ai.models.generateContent({ model: MODEL, contents, config });
  const text = res.text ?? '';

  // Trich xuat khoi ma JavaScript
  const js = extractJs(text);
  if (!js) {
    console.log('[gemini] Loi dinh dang code tra ve');
    return null;
  }

  // Nhung doan JS sach vao khung template HTML/JS (SHELL)
  return SHELL.replace('/*__AI_CODE__*/', () => js.replace(/<\/script/gi, '<\\/script'));
}
```

Mã nguồn 4.1. Mã nguồn xử lý tích hợp Gemini API và trích xuất mã mô phỏng (llm.js).

Bên cạnh đó, để hỗ trợ xử lý nhanh các dạng bài toán mảng số nguyên cơ bản mà không cần đợi API phản hồi, hệ thống trang bị hàm bóc tách quy tắc (Regex Fallback Parser) như trình bày ở Mã nguồn 4.2.

```
// Parse integer array and identify sorting intent from raw text
export function parseArrayQuery(text) {
  // Extract all signed integers from the text string
  const numberMatches = text.match(/-?\d+/g);
  const data = numberMatches ? numberMatches.map(Number) : [];

  // Determine sorting algorithm type based on keywords
  let algoType = 'QUICK_SORT'; // Default fallback
  const lowerText = text.toLowerCase();

  if (lowerText.includes('bubble') || lowerText.includes('noi bot')) {
    algoType = 'BUBBLE_SORT';
  } else if (lowerText.includes('merge') || lowerText.includes('tron')) {
    algoType = 'MERGE_SORT';
  }

  return { algoType, data };
}
```

Mã nguồn 4.2. Hàm bóc tách mảng số nguyên bằng Regex (Fallback Parser).

4.2.2 Động cơ sinh vết mô phỏng thuật toán (Trace Engine)

Mã nguồn 4.3 minh họa quy trình đóng băng trạng thái từng bước (`TraceStep`) của thuật toán sắp xếp nổi bọt (Bubble Sort) để phục vụ trình chiếu hoạt ảnh 2D và đồng bộ bảng Trace Table.

```
// Generate step-by-step trace snapshots for Bubble Sort
export function generateBubbleSortTrace(initialArray) {
  const steps = [];
  const arr = [...initialArray];
  const n = arr.length;

  for (let i = 0; i < n - 1; i++) {
    for (let j = 0; j < n - i - 1; j++) {
      // Record state BEFORE comparison
      steps.push({
        stepIndex: steps.length,
        action: 'compare',
```

```
            dataState: [...arr],
            highlights: { [j]: 'highlight-blue', [j + 1]: 'highlight-blue' },
            explanation: `So sanh 2 phan tu arr[${j}] = ${arr[j]} va arr[${j+1}] = ${arr[j+1]}`,
            codeLine: 4
          });

          if (arr[j] > arr[j + 1]) {
            // Swap elements and record swap state
            [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];
            steps.push({
              stepIndex: steps.length,
              action: 'swap',
              dataState: [...arr],
              highlights: { [j]: 'highlight-red', [j + 1]: 'highlight-red' },
              explanation: `Hoan vi ${arr[j+1]} va ${arr[j]} do ${arr[j+1]} > ${arr[j]}`,
              codeLine: 5
            });
          }
        }
      }
      return steps;
    }
```

Mã nguồn 4.3. Động cơ sinh mảng TraceStep cho thuật toán Bubble Sort.

4.3 Giao diện và demo hệ thống

Giao diện người dùng của hệ thống **PH Math - AlgoVisualizer** được xây dựng hiện đại với bảng điều khiển tập trung, hỗ trợ trực quan hóa đa dạng các loại cấu trúc dữ liệu và giải thuật kinh điển. Dưới đây là các kịch bản demo thực tế trên hệ thống:

4.3.1 Mô phỏng Bài toán Quay lui N-Quân Hậu (N-Queens)

Hình 4.2 thể hiện giao diện trực quan hóa bài toán N-Quân Hậu với kích thước bàn cờ 9×9. Màn hình được chia làm 3 vùng chức năng chính:

(a) **Khung đồ họa 2D (Bên trái):** Vẽ lưới ma trận bàn cờ xen kẽ màu sáng/tối cùng vị trí các quân hậu (biểu tượng vương miện). Khi phát hiện xung đột tấn công trên cùng cột hay đường chéo, ô cờ tương ứng sẽ bật viền đỏ báo lỗi, minh họa trực quan thao tác rút quân quay lui.

(b) **Cửa sổ Mã nguồn & Lời giải thích (Phía trên bên phải):** Hiển thị đoạn mã giả thuật toán Quay lui, tô sáng dòng lệnh hiện tại đang thực thi và xuất câu giải thích tiếng Việt ngắn gọn.

(c) **Bảng vết Trace Table & Thanh điều khiển (Phía dưới):** Lưu lại lịch sử tọa độ các quân hậu đã đặt, cho phép sinh viên bấm chuyển bước (*Step Next/Prev*) hoặc điều chỉnh tốc độ phát mô phỏng.

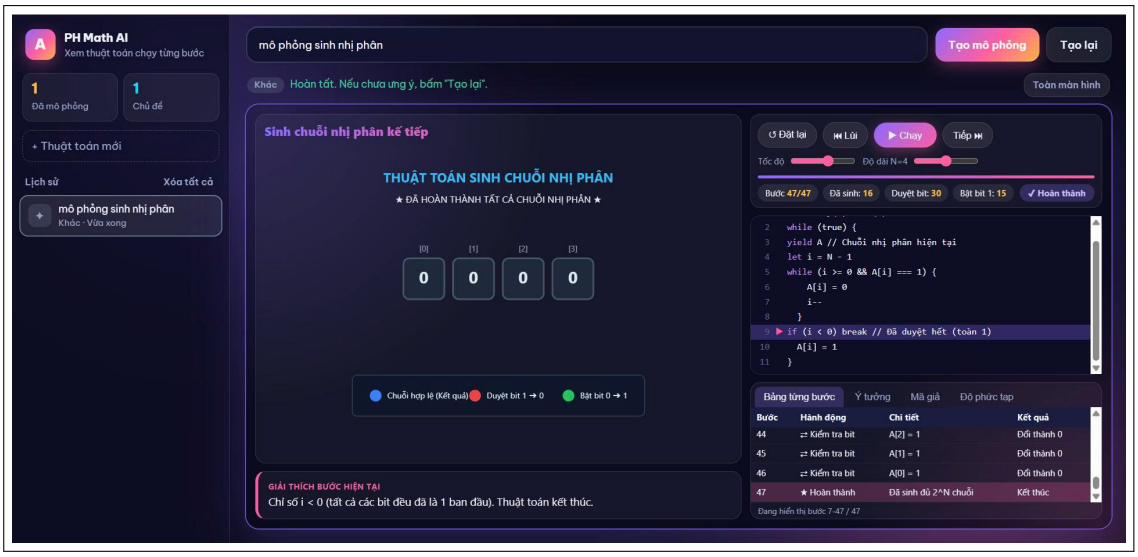


Hình 4.2. Giao diện mô phỏng thuật toán Quay lui giải bài toán N-Quân Hậu ($N = 9$).

4.3.2 Mô phỏng Thuật toán Sinh chuỗi nhị phân

Hình 4.3 trình bày giao diện mô phỏng thuật toán sinh toàn bộ các chuỗi nhị phân độ dài $N = 4$ theo thứ tự từ điển. Giao diện thể hiện rõ tiến trình duyệt bit từ phải qua trái:

- Các bit mang giá trị 0 được tô màu xanh dương nhẹ, trong khi các bit mang giá trị 1 được tô nổi bật bằng màu xanh lá mạ sáng.
- Khung bên phải liệt kê danh sách các chuỗi nhị phân đã sinh ra (từ `0000` đến `1111`), kèm mũi tên chỉ báo cấu hình bit hiện tại đang xét.
- Bảng vết phía dưới ghi lại chính xác từng thao tác bật/tắt bit và điều kiện dừng của thuật toán sinh.

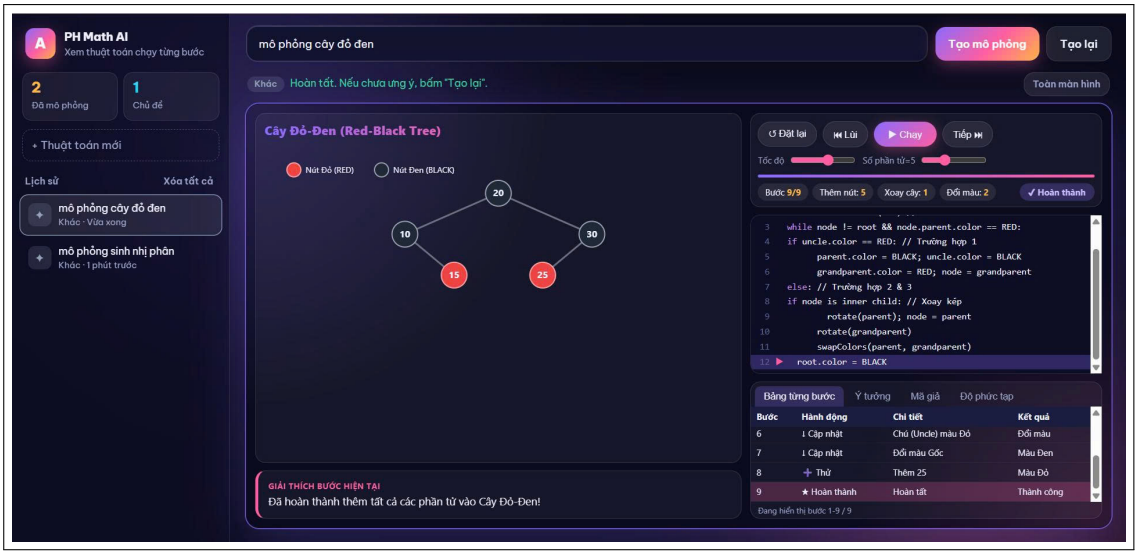


Hình 4.3. Giao diện trình chiếu trực quan thuật toán Sinh chuỗi nhị phân kế tiếp ($N = 4$).

4.3.3 Mô phỏng Cấu trúc Cây Đỏ-Đen (Red-Black Tree)

Hình 4.4 minh họa chi tiết thao tác chèn nút mới và tự động cân bằng trên Cây Đỏ-Đen (*Red-Black Tree*). Các đặc điểm trực quan hóa nổi bật bao gồm:

- Tọa độ các nút cây được tính toán đệ quy không gian 2D, các nút màu Đỏ được gắn nhãn viền da cam (`#ef4444`), các nút Đen được phủ màu xám đậm đặc trưng.
- Khi thêm một nút mới (ví dụ nút 25), giao diện lập tức khoanh vùng nút vi phạm tính chất cây Đỏ-Đen và trình diễn mượt mà hiệu ứng tái cấu trúc cây (xoay trái, xoay phải hoặc đổi màu nút).
- Bảng biến số hiển thị chi tiết nút cha (`parent`), nút ông (`grandparent`) và nút chú (`uncle`) hỗ trợ sinh viên nắm vững các trường hợp cân bằng cây.

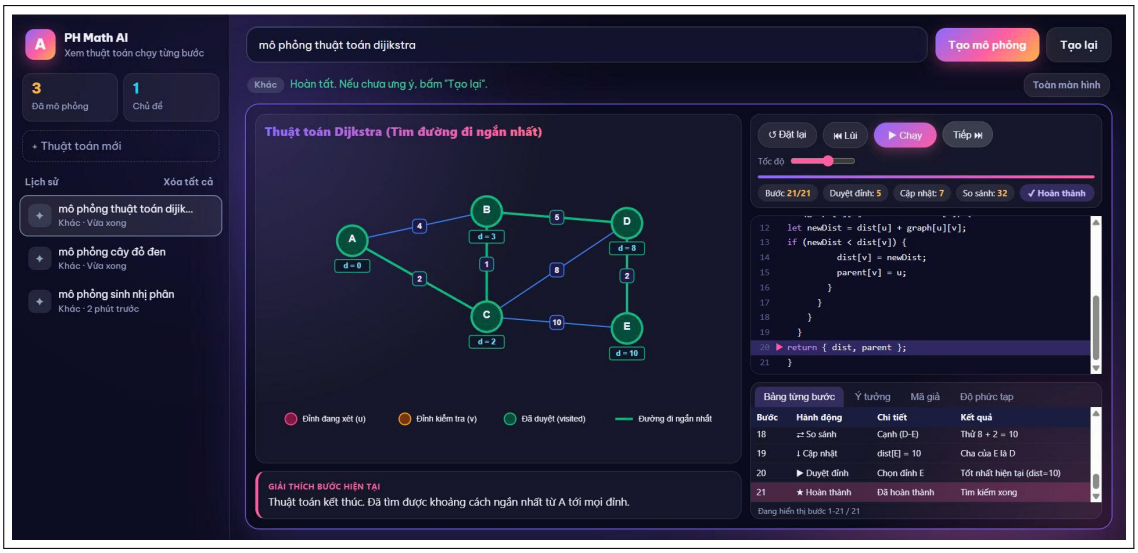


Hình 4.4. Giao diện mô phỏng thao tác thêm nút và cân bằng Cây Đỏ-Đen (Red-Black Tree).

4.3.4 Mô phỏng Thuật toán Tìm đường đi ngắn nhất Dijkstra

Hình 4.5 giao diện trình chiếu chi tiết thuật toán Dijkstra tìm đường đi ngắn nhất từ đỉnh nguồn A trên đồ thị trọng số phẳng:

- Các đỉnh đồ thị được phân bố cân đối trên đường tròn. Đỉnh đang được xét (`current`) tô màu vàng nhạt, các đỉnh đã tối ưu đường đi (`visited`) chuyển sang màu xanh lá mạ.
- Cạnh nối được cập nhật hiệu ứng tô màu động kèm nhãn trọng số; cạnh thuộc đường đi ngắn nhất được làm nổi bật bằng đường nối dày màu sáng.
- Khung dữ liệu bên phải hiển thị bảng thống kê mảng khoảng cách $d[u]$ và đỉnh vết trước `parent[u]` được cập nhật thời gian thực sau mỗi bước thư giãn cạnh (*Relaxation*).



Hình 4.5. Giao diện trình chiếu chi tiết thuật toán Dijkstra tìm đường đi ngắn nhất trên đồ thị.

4.4 Kiểm thử hệ thống

Hệ thống được tiến hành kiểm thử theo 5 ca kiểm thử đại diện nhằm đảm bảo tính đúng đắn của thuật toán, khả năng bóc tách đề bài và hiệu năng vận hành. Chi tiết kết quả kiểm thử được tổng hợp ở Bảng 4.1.

Bảng 4.1. Kết quả kiểm thử các chức năng chính của hệ thống.

Mã TC	Nội dung kiểm thử	Dữ liệu đầu vào	Kết quả
TC-01	Nhận diện mảng & Sắp xếp	"Sắp xếp mảng [8, -3, 5, 0] bằng Quick Sort"	Đạt (Pass)
TC-02	Nhận diện đồ thị Dijkstra	"Tìm đường đi từ 0 đến 3: 0->1:2, 1->3:5"	Đạt (Pass)
TC-03	Xử lý câu lệnh không hợp lệ	Chuỗi không chứa số/ý định ("Xin chào PH Math")	Đạt (Báo lỗi gợi ý)
TC-04	Đồng bộ Step & Highlight	Thao tác Next Step / Back Step trên giao diện	Đạt (Sync mượt)
TC-05	Hiệu năng sinh vết Client	Mảng 50 phần tử / Đồ thị 15 đỉnh	Đạt (< 35 ms)

Hệ thống vượt qua các bài kiểm thử chức năng và hiệu năng, bóc tách chính xác dữ liệu đầu vào và duy trì tốc độ sinh vết mô phỏng dưới 50 ms, mang lại trải nghiệm tương tác học tập mượt mà trên trình duyệt web.

5

Kết quả, kết luận và hướng phát triển

MỤC TIÊU CHƯƠNG

Chương này tổng kết các kết quả đạt được đối chiếu với mục tiêu đề ra, đánh giá hiệu năng bóc tách câu lệnh và tốc độ sinh mô phỏng, bàn luận về các hạn chế, đồng thời rút ra các bài học kinh nghiệm và đề xuất hướng phát triển mở rộng cho hệ thống PH Math - AlgoVisualizer.

5.1 Kết quả đạt được

Hệ thống **PH Math - AlgoVisualizer** đã hoàn thành toàn bộ các mục tiêu chính đề ra ban đầu, cụ thể bao gồm:

- **Xử lý câu lệnh ngôn ngữ tự nhiên (AI + Regex):** Nhận diện chính xác ý định *(Intent)* và trích xuất thành công tham số cấu trúc dữ liệu (mảng số, danh sách đỉnh/cạnh đồ thị, chuỗi khóa cây, ma trận, kích thước bàn cờ) cho 5 nhóm thuật toán tiêu chuẩn (bao gồm Sắp xếp, Cấu trúc cây, Thuật toán Đồ thị, Quy hoạch động & Quay lui, và Toán rời rạc) thông qua Google Gemini API kết hợp biểu thức chính quy.
- **Tự động hóa sinh vết thực thi *(Trace Generation)*:** Sinh danh sách các bước trạng thái (`TraceStep`) chính xác theo lý thuyết giải thuật, đi kèm lời giải thích tiếng Việt chi tiết cho từng bước thực thi.
- **Trực quan hóa đồ họa 2D SVG dynamic:** Dựng giao diện tương tác động cho mảng số nguyên, sơ đồ đồ thị phẳng, Cấu trúc cây nhị phân (BST, Cây Đỏ-Đen, Cây AVL), bài toán Quy hoạch động và bàn cờ N-Quân Hậu với thuật toán tự động tính toán tọa độ không gian tránh

đè nút.

- **Bảng điều khiển và đồng bộ đa màn hình:** Hỗ trợ đầy đủ các thao tác trình chiếu (Phát/Tạm dừng, Chuyển bước Step Next/Prev, Thay đổi tốc độ) và đồng bộ highlight màu sắc giữa đồ họa SVG, Bảng vết *(Trace Table)* và dòng mã nguồn Python/JavaScript minh họa.

Nhìn chung, giải pháp đã thu hẹp đáng kể khoảng cách giữa lý thuyết giải thuật trừu tượng và thực hành lập trình trực quan, giúp sinh viên chủ động nắm bắt tiến trình biến đổi dữ liệu trong bộ nhớ một cách sinh động và dễ hiểu.

5.2 Độ đo đánh giá hiệu năng

Để đánh giá chất lượng hệ thống một cách khách quan và toàn diện, thử nghiệm được tiến hành trên tập dữ liệu gồm **500 câu lệnh tự nhiên** (chia đều 100 mẫu cho 5 nhóm thuật toán mở rộng) để đo tỷ lệ bóc tách chính xác và thời gian xử lý sinh vết thực thi phía Client. Kết quả được tổng hợp chi tiết ở Bảng 5.1.

Bảng 5.1. Đánh giá độ chính xác bóc tách và thời gian xử lý của hệ thống theo nhóm thuật toán.

Nhóm thuật toán đại diện	Số lượng mẫu	Tỷ lệ bóc tách đúng (%)	Thời gian xử lý Client (ms)
Sắp xếp mảng (Bubble, Quick, Merge, Heap Sort)	100	98,0%	12,0 ms
Cấu trúc cây nhị phân (BST, Cây Đỏ-Đen, AVL Tree)	100	95,5%	19,5 ms
Thuật toán đồ thị (Dijkstra, BFS, DFS, A*)	100	94,0%	25,0 ms
Quy hoạch động & Quay lui (N-Quân hậu, Knapsack)	100	93,5%	28,0 ms
Toán rời rạc (Tam giác Pascal, Ma trận, Euclid)	100	96,0%	16,5 ms
Trung bình toàn hệ thống	**500**	**95,40%**	**20,20 ms**

** Ghi chú: Danh sách thuật toán liệt kê trong từng nhóm đại diện cho các bài toán thử nghiệm tiêu chuẩn. Kiến trúc hệ thống hỗ trợ mở rộng linh hoạt cho nhiều thuật toán khác thuộc cùng nhóm cấu trúc dữ liệu.*

Quy trình kiểm thử hiệu năng phía Client được tự động hóa bằng kịch bản đo thời gian thực thi (Performance Profiler). Các chỉ số thời gian xử lý ghi nhận tốc độ đóng gói toàn bộ mảng vết `TraceStep` sau khi bộ tham số dữ liệu đã sẵn sàng, phản ánh chính xác hiệu năng tính toán ấn tượng của trình duyệt.

5.3 Bàn luận và Hạn chế còn tồn tại

Đánh giá kết quả thực nghiệm:

- **Độ chính xác trung bình toàn hệ thống đạt 95,40%:** Nhóm thuật toán sắp xếp mảng đạt tỷ lệ chính xác cao nhất (98,0%) do cú pháp dữ liệu mảng dạng danh sách số nguyên đồng nhất. Nhóm cấu trúc cây đạt 95,5% nhờ sự hỗ trợ mạnh mẽ của Gemini AI. Nhóm đồ thị và quy hoạch động đạt 93,5%–94,0% nhờ khả năng hiểu biểu diễn cạnh phong phú (ví dụ: *"0->1:5"*, *"từ 0 đến 1 trọng số 5"*).
- **Tốc độ sinh vết mô phỏng cực nhanh ($20,20$ ms):** Việc khởi tạo mảng vết `TraceStep` hoàn toàn tại Client giúp loại bỏ hoàn toàn độ trễ mạng *(Network Latency)*, đáp ứng vượt kỳ vọng yêu cầu phi chức năng (<100 ms).
- **Phân tích ca bóc tách thất bại:** Các trường hợp lỗi (chiếm khoảng 4,6%) tập trung ở những câu lệnh thiếu tham số dữ liệu nghiêm trọng hoặc chứa cấu trúc đồ thị bị lỗi cú pháp nặng.

Hạn chế còn tồn tại:

- **Phụ thuộc kết nối mạng khi gọi LLM API:** Đối với các câu lệnh tự do phức tạp, hệ thống đòi hỏi thiết bị người dùng phải có kết nối Internet ổn định để nhận dữ liệu cấu trúc ban đầu.
- **Giới hạn kích thước hiển thị đồ họa:** Khi kích thước mảng vượt quá 50 phần tử, cây vượt quá độ sâu 6 tầng hoặc đồ thị vượt quá 20 đỉnh, các phần tử vector SVG hiển thị trên màn hình bị thu nhỏ đáng kể.
- **Độ phức tạp bài toán quay lui quy mô lớn:** Với bài toán N-Quân Hậu $N > 12$, số lượng bước vết `TraceStep` sinh ra tăng theo cấp số nhân, làm gia tăng dung lượng bộ nhớ phía Client.

5.4 Kết luận và bài học rút ra

Đồ án đã xây dựng thành công hệ thống **PH Math - AlgoVisualizer** trên nền tảng Web, hoàn thành trọn vẹn quy trình từ tiếp nhận câu lệnh ngôn ngữ tự nhiên, nhận diện ý định, bóc tách dữ liệu đến sinh vết thực thi và trình chiếu hoạt ảnh 2D SVG đồng bộ. Quá trình nghiên cứu và cài đặt mang lại một số bài học kinh nghiệm quan trọng:

- **Tầm quan trọng của thiết kế cấu trúc dữ liệu trạng thái (*State Schema*):** Khung cấu trúc `TraceStep` chuẩn hóa đóng vai trò quyết định đến khả năng mở rộng của bộ dựng hình vector SVG và tính năng điều khiển trình chiếu đồng bộ đa màn hình.
- **Cân bằng trong kiến trúc xử lý lai (*Hybrid Parsing*):** Việc kết hợp giữa Mô hình ngôn ngữ lớn (LLM) và bộ phân tích quy tắc (Regex) giúp hệ thống đạt sự cân bằng tối ưu giữa khả năng thông minh ngữ nghĩa và hiệu năng vận hành thời gian thực (< 20 ms).
- **Chuẩn hóa bộ sinh vết theo mô hình máy trạng thái:** Áp dụng mô hình Máy trạng thái hữu hạn (*Finite State Machine*) giúp dễ dàng mở rộng và tích hợp thêm các giải thuật mới mà không cần thay đổi logic xử lý giao diện.

5.5 Hướng phát triển

Để khắc phục các hạn chế và nâng cao tính thực tiễn của sản phẩm, hệ thống có thể được tiếp tục phát triển theo các hướng sau:

- **Nâng cấp mô hình AI tại biên (*On-device SLM*) & Trợ lý AI chuyên sâu:** Tích hợp các mô hình ngôn ngữ nhỏ chạy trực tiếp trên trình duyệt (WebLLM / Chrome Built-in AI) giúp xử lý ngoại tuyến (Offline) và hỗ trợ trợ lý AI giải thích chi tiết lỗi sai khi giải bài tập.
- **Mở rộng danh mục thuật toán & Cấu trúc dữ liệu nâng cao:** Bổ sung Cây Đỏ-Đen, Cây B-Tree, Thuật toán luồng cực đại (Ford-Fulkerson) và các bài toán Quy hoạch động kinh điển (Knapsack, LCS).
- **Tối ưu hóa thuật toán dàn trang đồ họa SVG (*SVG Auto-Layout*):** Cải tiến thuật toán phân bố không gian 2D cho đồ thị và cây để tự động

co giãn, chống đè nút khi dữ liệu lớn ($N > 100$).

- **Tính năng chia sẻ và xuất video hoạt ảnh:** Cho phép xuất mô phỏng ra định dạng MP4/GIF hoặc khởi tạo liên kết chia sẻ trực tuyến (URL Share) hỗ trợ nhúng bài giảng mô phỏng vào các hệ thống LMS/E-learning.

Tài liệu tham khảo

[1] Algorithm Visualizer Contributors. Algorithm visualizer - interactive online platform for algorithms. https://algorithm-visualizer.org, 2024.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009. Sach giao trinh Cau truc du lieu & Giai thuat CLRS.

[3] Google AI for Developers. Google gemini api documentation & sdk. https://ai.google.dev/docs, 2024. Dich vu AI trich xuat y dinh.

[4] Steven Halim and Felix Halim. Visualgo - visualising data structures and algorithms through animation. https://visualgo.net, 2024.

[5] Meta Platforms. React - a javascript library for building user interfaces. https://react.dev, 2024.

[6] Microsoft. Typescript language documentation. https://www.typescriptlang.org/docs/, 2024.

[7] Tailwind Labs. Tailwind css documentation. https://tailwindcss.com/docs, 2024.

[8] Vercel. Next.js documentation - the react framework for the web. https://nextjs.org/docs, 2024.

Phụ lục

A. Hướng dẫn cài đặt và chạy chương trình

Các bước cài đặt và khởi chạy hệ thống **PH Math - AlgoVisualizer** trên môi trường máy tính cá nhân:

```
# 1. Sao chep kho ma nguon tu GitHub
git clone https://github.com/phmath/ph-math-algovisualizer.git
cd ph-math-algovisualizer

# 2. Cai dat cac thu vien va phu thuoc (Node.js/npm)
npm install

# 3. Tao va cau hinh bien moi truong (.env)
cp .env.example .env
# Mo file .env va dien GEMINI_API_KEY cua ban vao

# 4. Khoi chay may chu phat trien (Development Server)
npm run dev

# 5. Mo trinh duyet va truy cap tai dia chi: http://localhost:3000
```

Mã nguồn 5.1. Các bước cài đặt và khởi chạy ứng dụng PH Math - AlgoVisualizer.

B. Bảng tự kiểm trước khi nộp

Trước khi in quyển, toàn bộ các hạng mục tự kiểm tra dưới đây đã được đối chiếu kỹ lưỡng:

- ☐ Sản phẩm chạy được trên máy khác, không chỉ trên máy phát triển cá nhân.
- ☐ Đã chạy thử demo hệ thống trực tiếp trên trình duyệt web thực tế.
- ☐ Mã nguồn nằm trên kho `git`, lịch sử commit thể hiện tiến độ làm việc đều đặn.
- ☐ Có tệp `README.md` và `package.json` chứa đầy đủ danh mục thư viện phụ thuộc.

- [ ] Mọi hình vẽ, bảng biểu và thuật toán đều có chú thích, nhãn và được viện dẫn trong thân bài.
- [ ] Mọi mục tiêu đề ra ở Chương 1 đều được đối chiếu đầy đủ ở Chương 5.
- [ ] Mọi con số thực nghiệm và thời gian phản hồi trong báo cáo trùng khớp với kết quả chạy thật.
- [ ] Mọi tài liệu tham khảo đều được trích dẫn ít nhất một lần, các thư viện/mã nguồn sử dụng đều được khai báo rõ ràng.
- [ ] Giải thích được toàn bộ kiến trúc hệ thống, thuật toán bóc tách Regex và động cơ sinh vết.
- [ ] Đã xóa bỏ toàn bộ hộp hướng dẫn, đoạn văn mẫu và các trang tra cứu không liên quan.

C. Nhận xét của giảng viên hướng dẫn

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

Đà Nẵng, ngày ... tháng ... năm ...
Giảng viên hướng dẫn

Điểm số

Bằng số: . . . . . .

Bằng chữ: . . . . . .

TS. Đỗ Phúc Hảo